



香港科技大學(廣州)
香港
科技大学 (广州)



香港
香港科学大学
和技术

AIAA 2205: 人工智能导论

戴恩艳

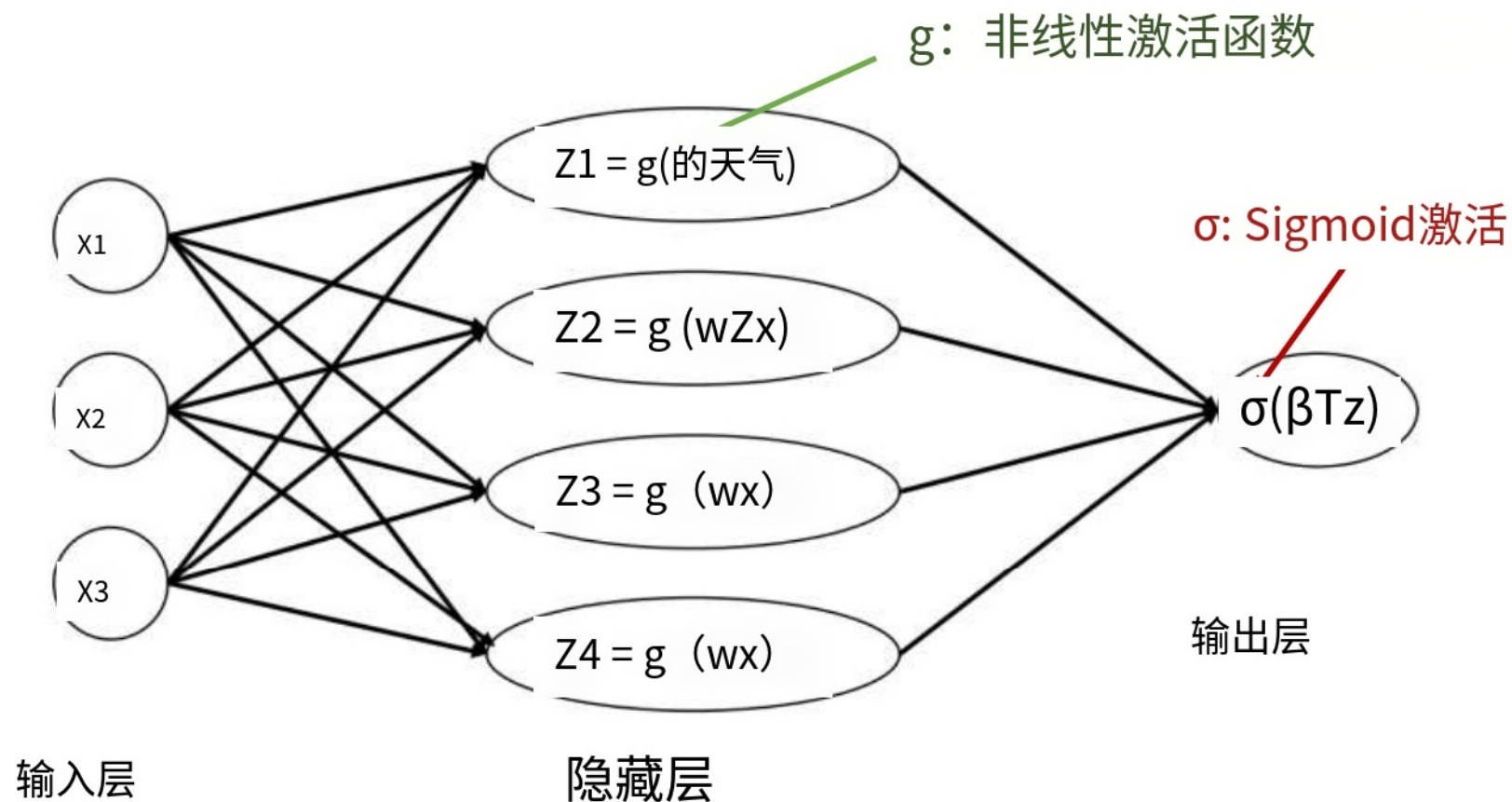
香港科技大学 (广州)

多层感知器

多层感知机由多层神经网络组成

对于二分类问题: $F(X) = \sigma(\beta^T G(WX))$

多层感知机可以堆叠许多隐藏层

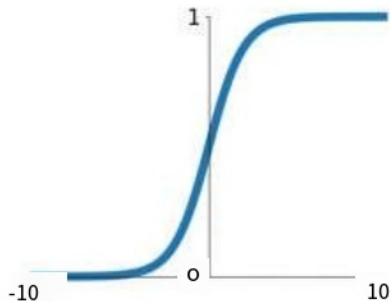


激活函数

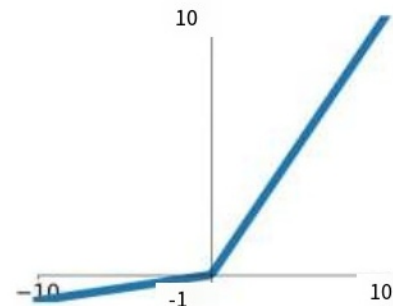
ReLU 对于大多数问题来说都是一个很好的默认选择!

乙状结肠

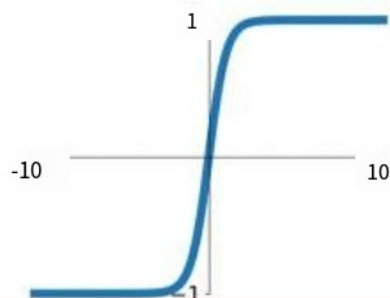
$$\sigma(x) = \frac{1}{1+e^{-x}} =$$



漏水的ReLU
马克斯(0.1 x, x)



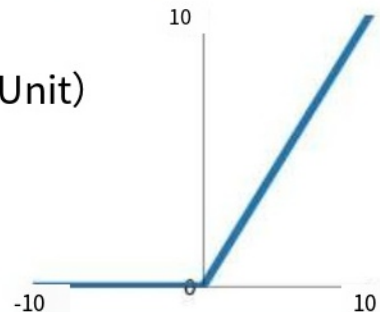
腥臭(x)



Maxout
 $\text{Max}(wx + b1, w + b2)$

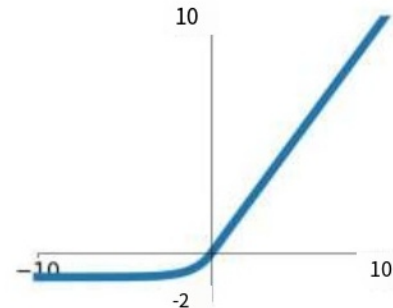
线性整流函数 (Rectified Linear Unit)

马克斯(0,x)

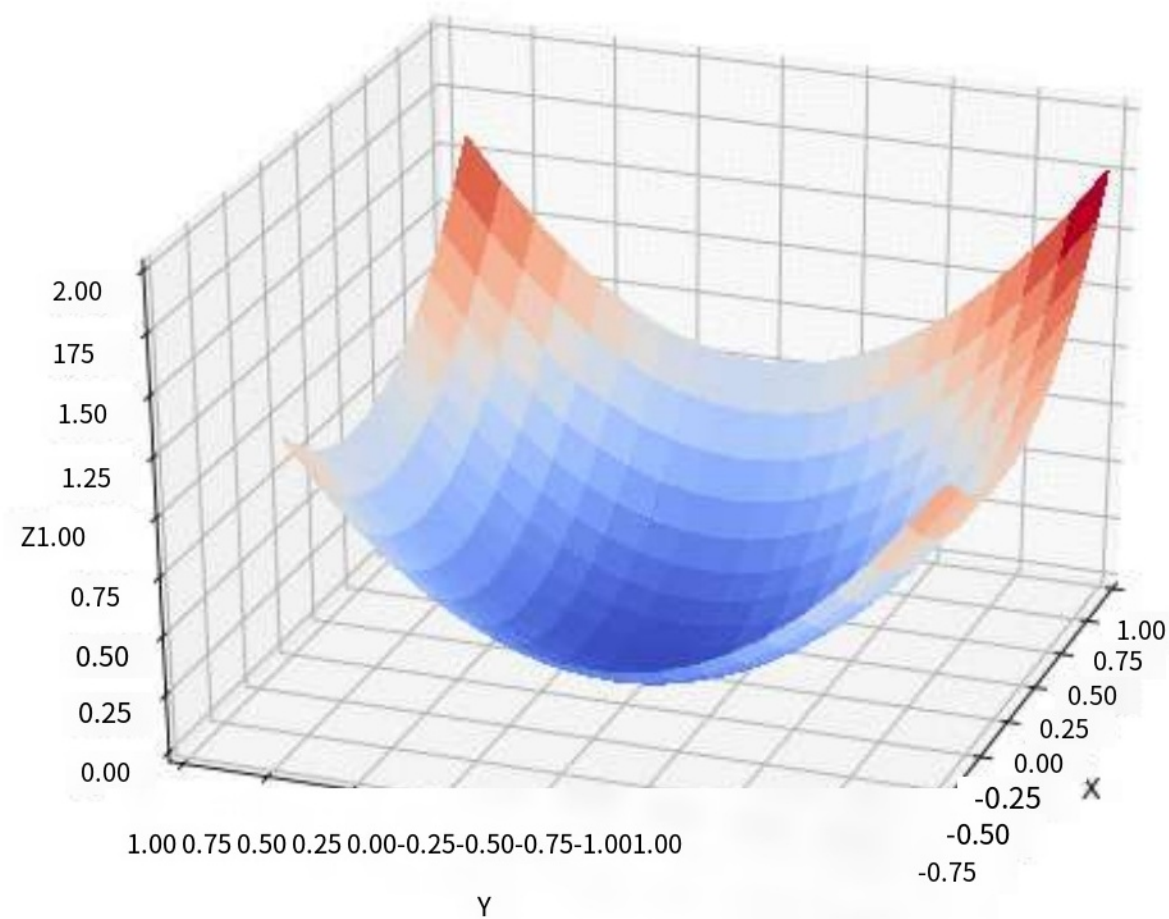


生命

$$\begin{cases} \text{我} & x \geq 0 \\ 1(e) & x < 0 \end{cases}$$



梯度下降



思路：沿着导致损失减少最迅速的方向前进。

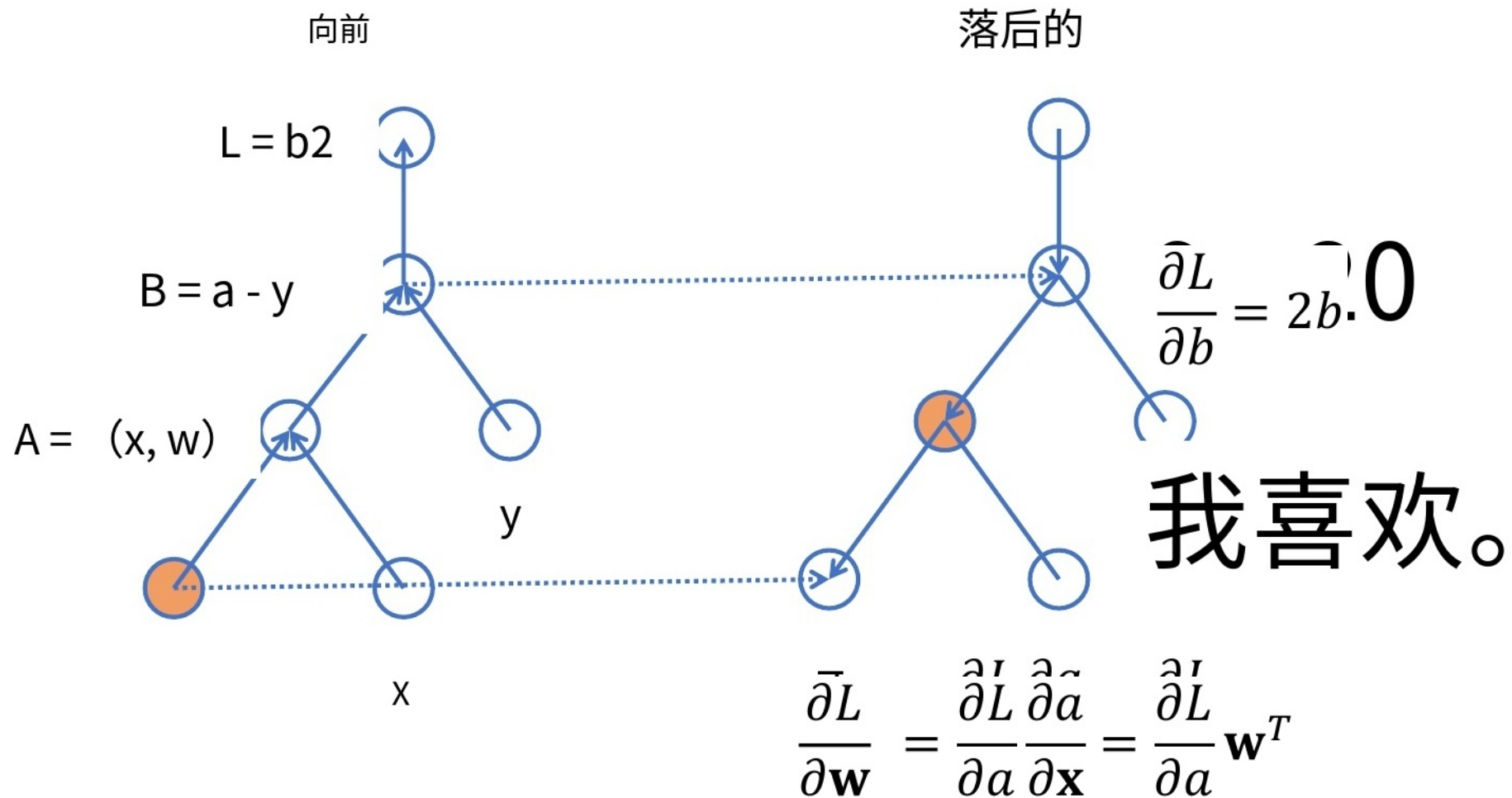
$$\mathbf{W} \leftarrow \mathbf{W} - \lambda \frac{\partial L}{\partial \mathbf{W}}$$

步长

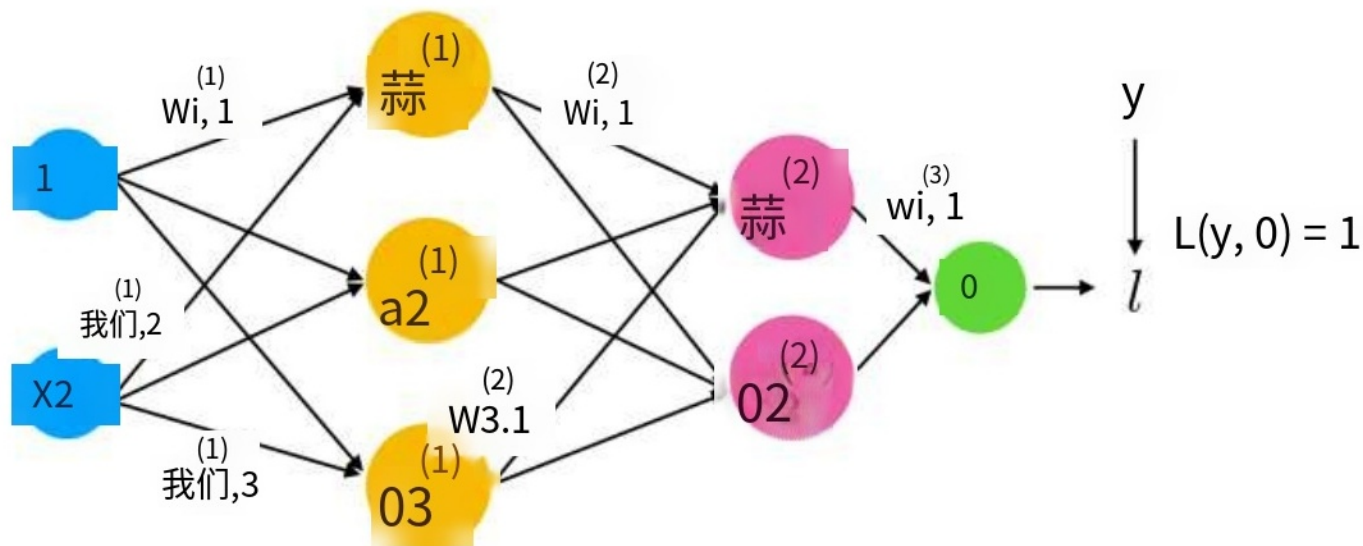
梯度

反向累加

假设 $L = (x, w) - y)^2$



梯度消失问题



假设应用了 S 型激活函数:

$$\frac{\partial l}{\partial w_{1,1}^{(1)}} = \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_1^{(2)}} \cdot \frac{\partial a_1^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}} + \frac{\partial l}{\partial o} \cdot \frac{\partial o}{\partial a_2^{(2)}} \cdot \frac{\partial a_2^{(2)}}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{1,1}^{(1)}}$$

$$a_1^{(2)} = \sigma(z_1^{(2)}) = \frac{1}{1 + \exp(-z_1^{(2)})}$$

$$z_1^{(2)} = w_{1,1}^{(2)} a_1^{(1)} + w_{2,1}^{(2)} a_2^{(1)} + w_{3,1}^{(2)} a_3^{(1)}$$

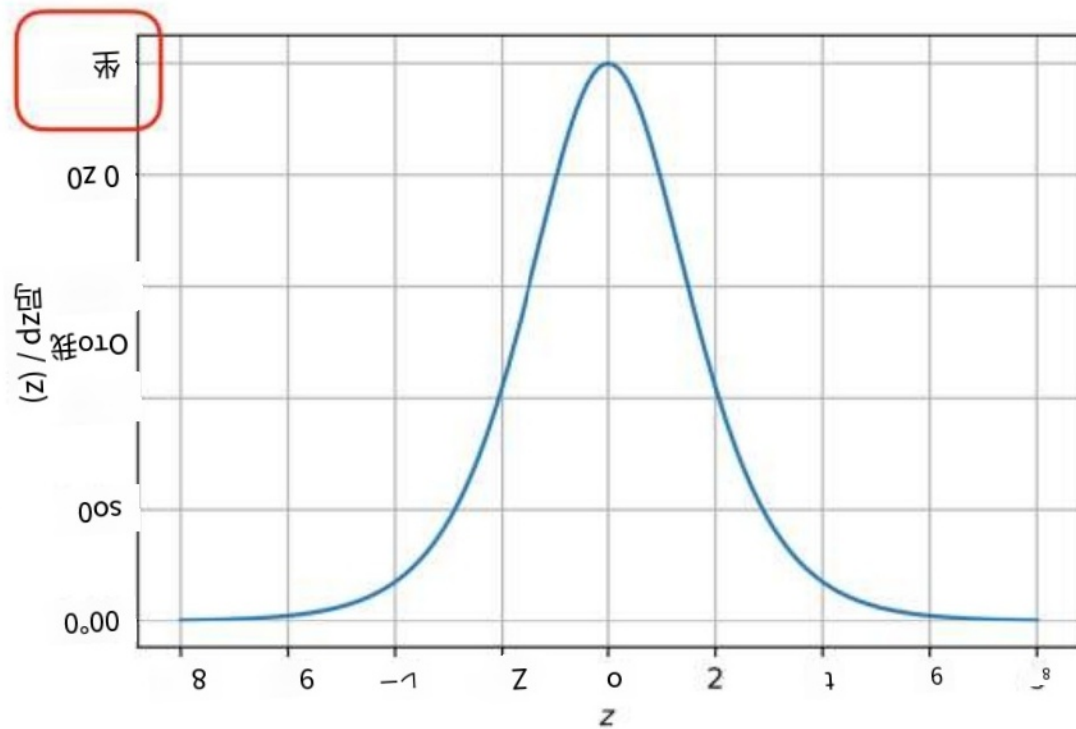
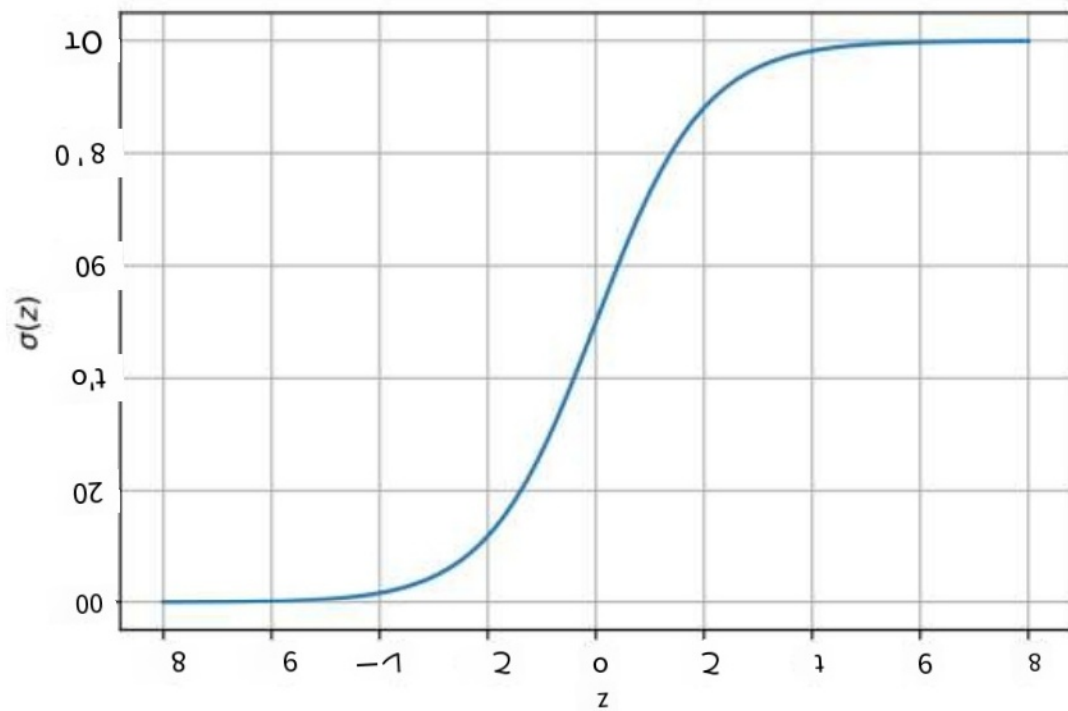
梯度消失问题

S 型函数的最大梯度为 **0.25**。

假设使用 10 层 S 型函数: $0.25^{10} \approx 10^{-6}$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)(1 - \sigma(z))$$



具有相同方差的随机初始化

```
dims = [4096] * 7      Forward pass for a 6-layer  
                        net with hidden size 4096  
hs = []  
x = np.random.randn(16, dims[0])  
for Din, Dout in zip(dims[:-1], dims[1:]):  
    W = 0.01 * np.random.randn(Din, Dout)  
    x = np.tanh(x.dot(W))  
    hs.append(x)
```

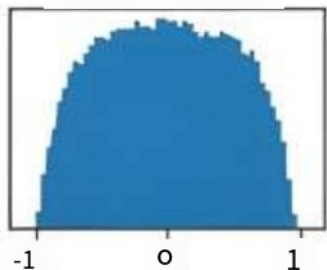
所有深度网络层的激活值
都趋于零

关于梯度呢？

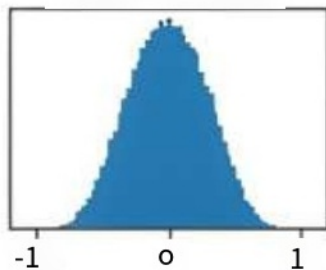
$$x_{n+1} = \tanh(z_{n+1}) \quad z_{n+1} = w_n x_n + b$$

$$\frac{dx_{n+1}}{dw_n} = \frac{dx_{n+1}}{dz_{n+1}} \cdot x_n \approx 0$$

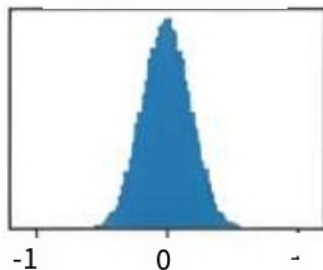
Layer 1
mean=-0.00
std=0.49



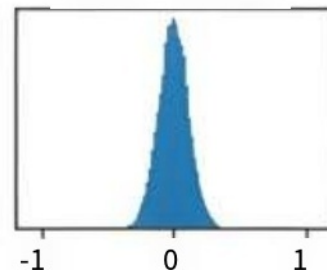
第二层均值
=0.00 std=0.29



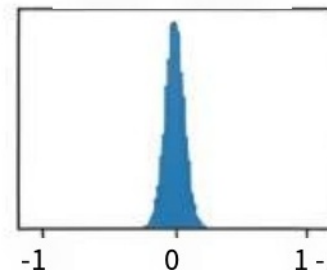
第3层均值
=0.00 std=0.18



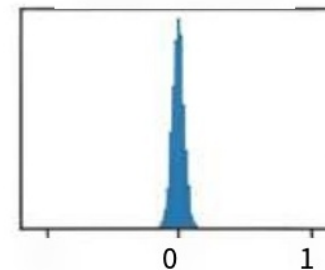
第4层均值
=-0.00 std=0.11



第5层均值
=-0.00 std=0.07



第6层均值
=0.00 std=0.05



优化器

梯度下降

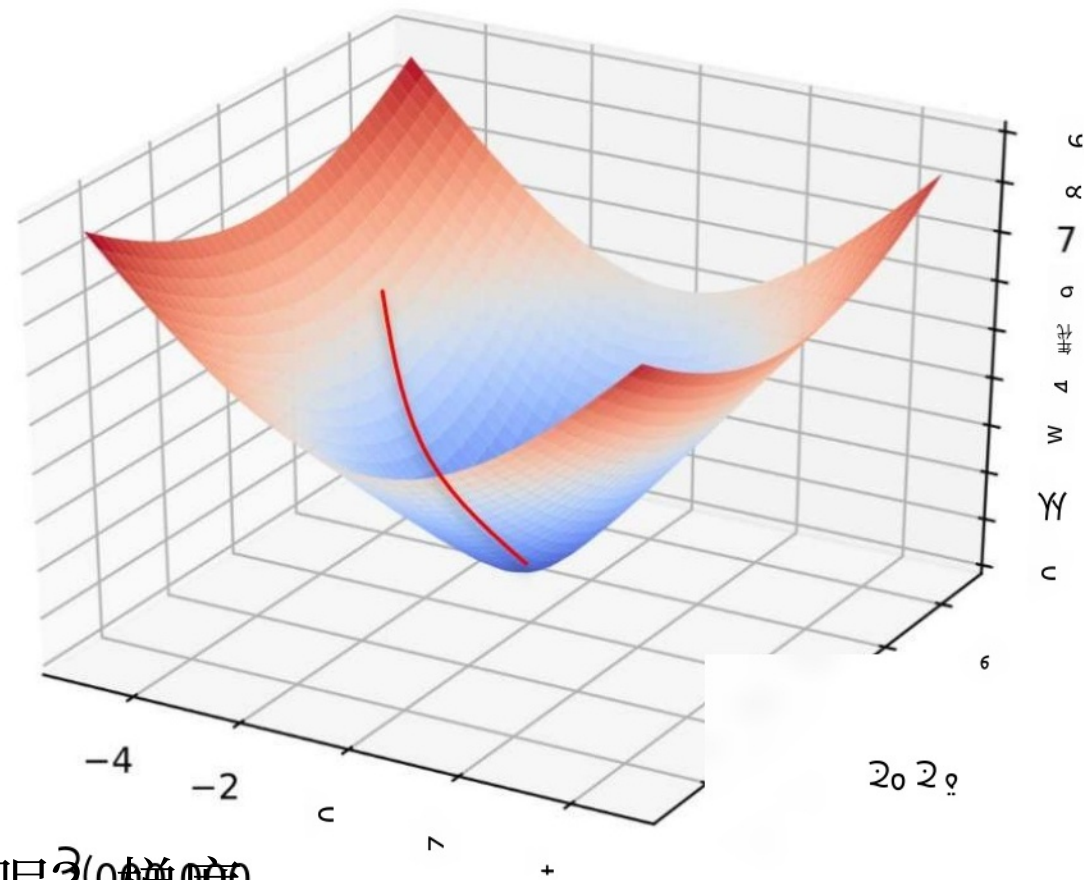
损失函数

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N l(x_i, y_i, \theta)$$

对于 $t = 1, \dots, T$:

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \alpha \nabla_{\theta} J(\theta)$$

如果我们有 100 万个训练样本 ($N > 100$ 万) 呢? 梯度下降法的计算量太大了。



随机梯度下降

对于 $t = 1, \dots, T$:

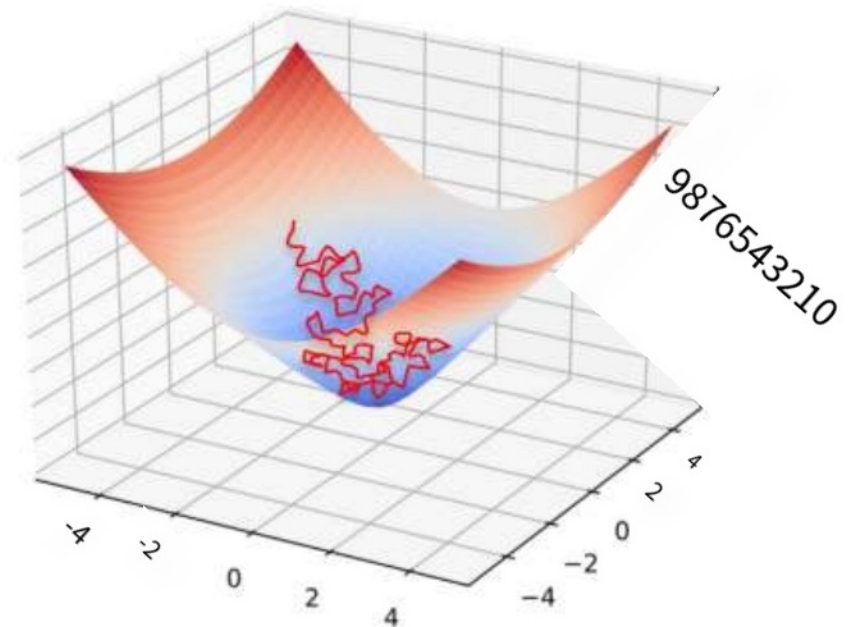
从样本中选取 K 个数据点作为一批数据集 D

计算批次上的损失 $J_{\theta} = \frac{1}{K} \sum_{X^i \in D} L(X^i, Y^i, \theta)$

更新参数 $\theta^{(T)} \leftarrow \theta^{(T-1)} - \alpha \nabla_{\theta} J_{\theta}$

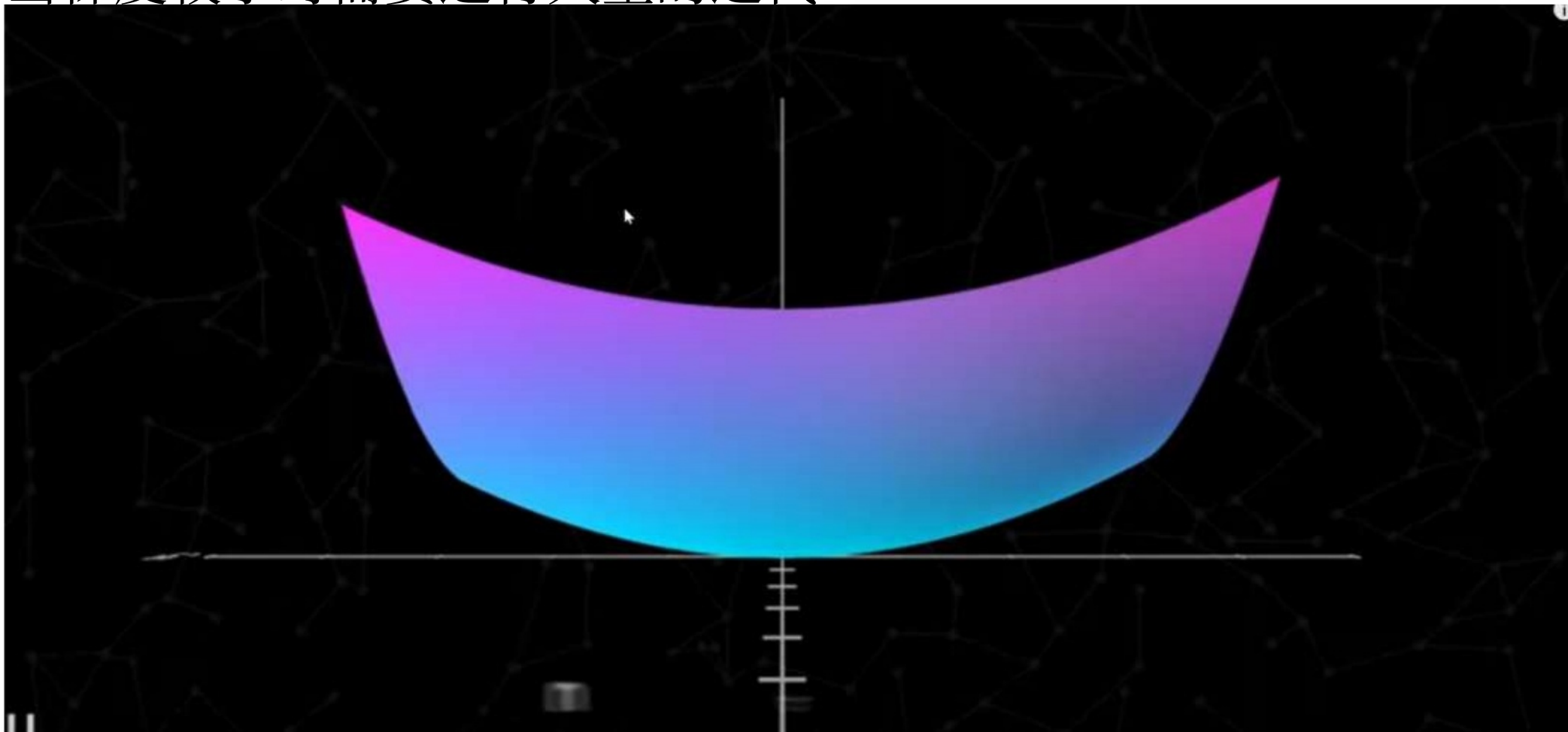
批量大小可以远小于训练数据的大小。

- 32、64、128、256、512、1024、……
- 批量越大越好



优化：使用随机梯度下降法 (SGD) 的问题 #1

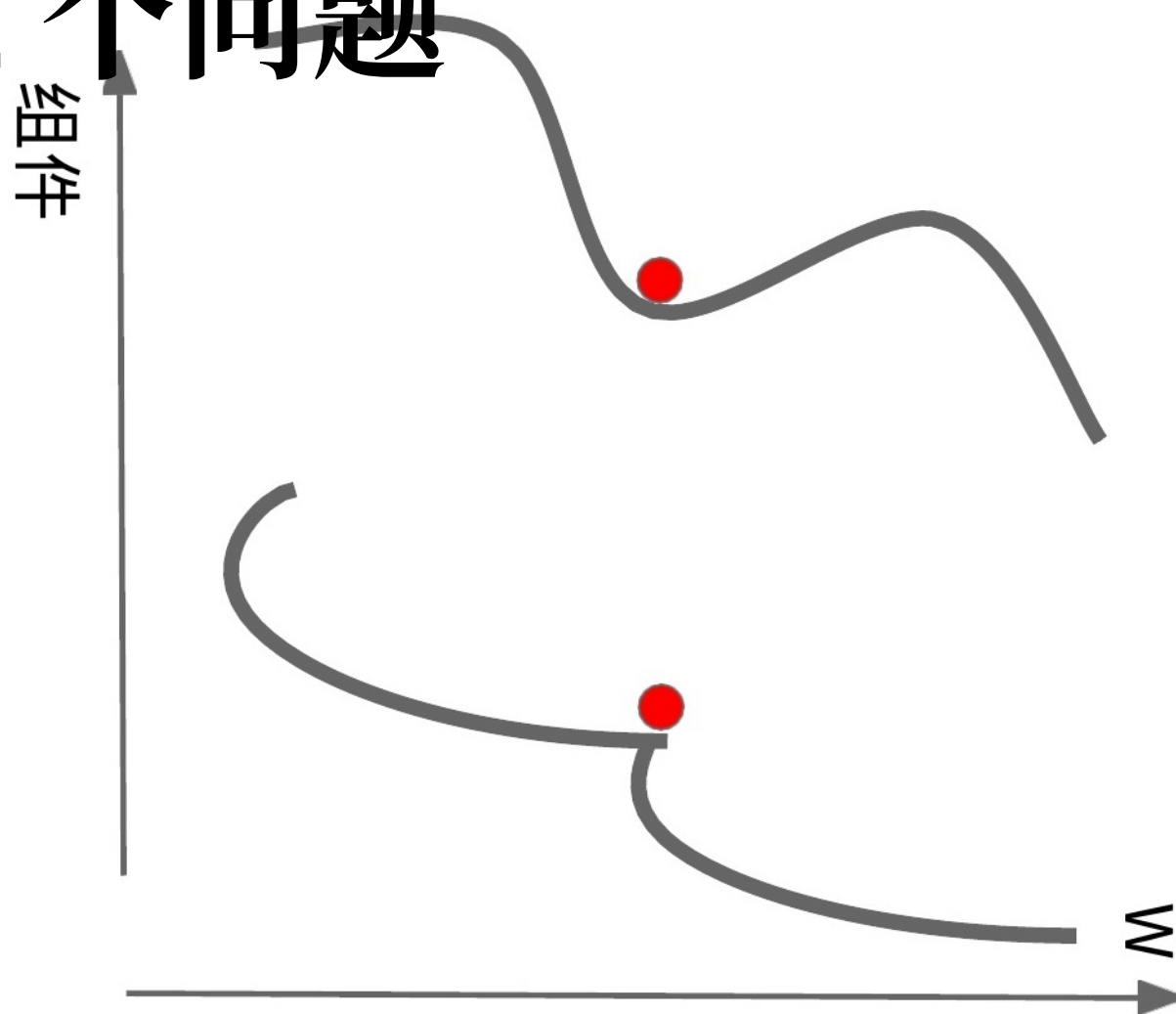
当梯度较小时需要进行大量的迭代



优化：使用随机梯度下降法（SGD）的第2个问题

如果损失函数存在局部最小值或鞍点会怎样？

梯度为零，梯度下降法陷入停滞

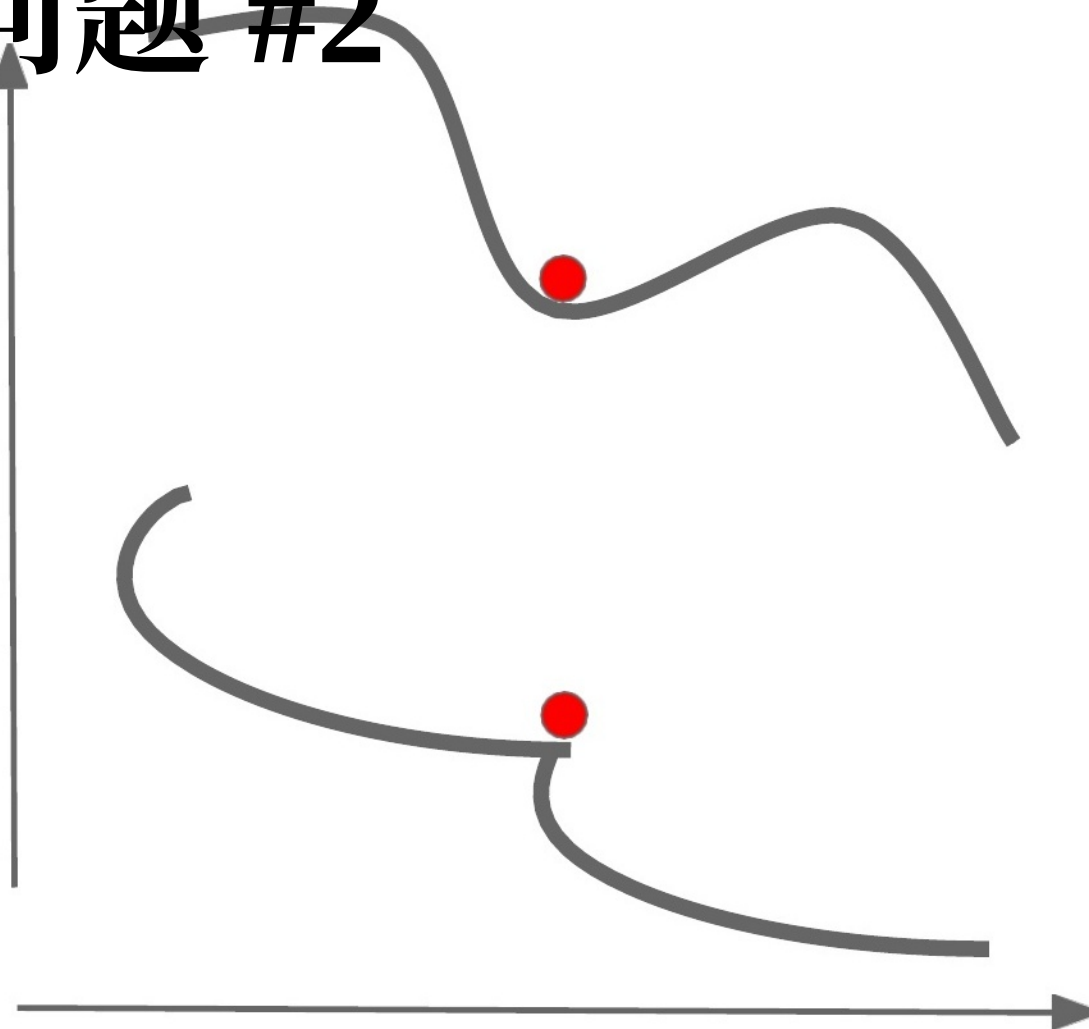


优化：使用随机梯度下降法（SGD）解决的问题 #2

如果损失函数存在局部最小值或鞍点会怎样？

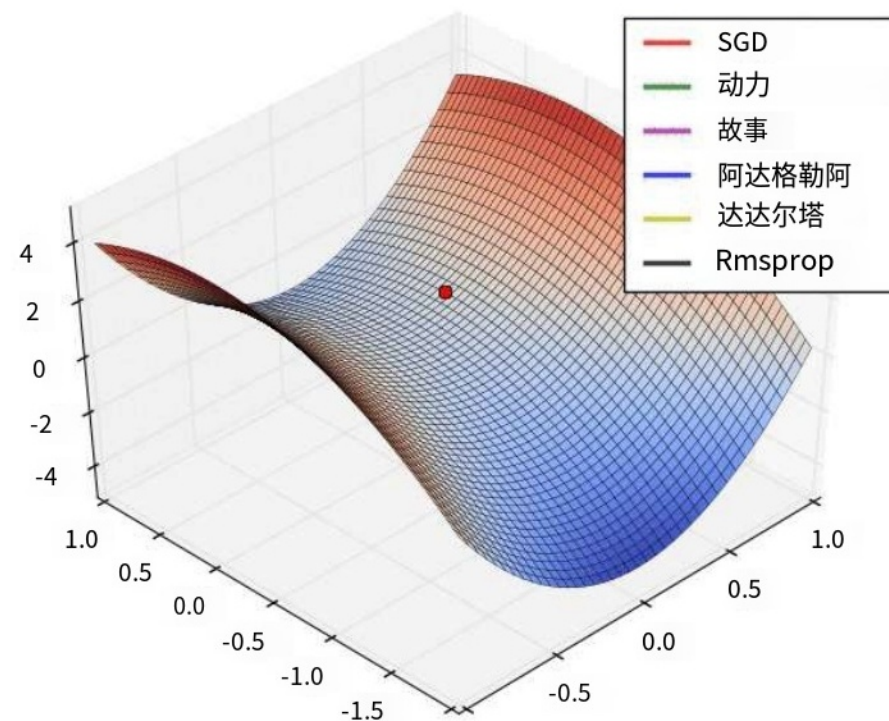
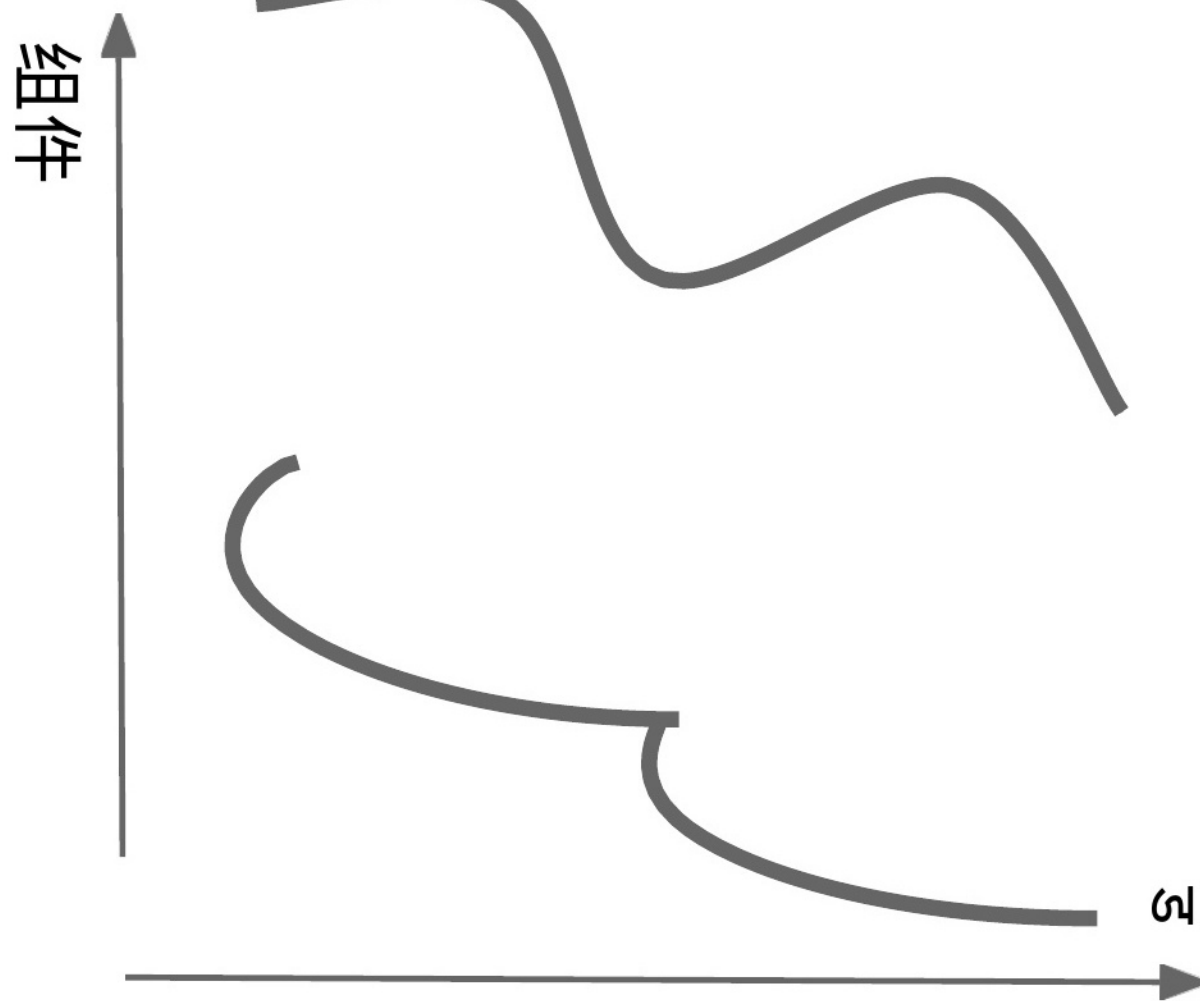
在高维空间中，
鞍点要常见得多

组件



优化：问题 #2 使用随机梯度

下降法时陷入局部最小值或鞍点



解决方案有哪些？

1.1核心理念

动量通过积累过去的梯度来抑制陡峭或嘈杂区域的振荡，从而加速梯度下降。它引入了一个“速度”项，使优化路径变得平滑。

1.2数学公式

更新规则:

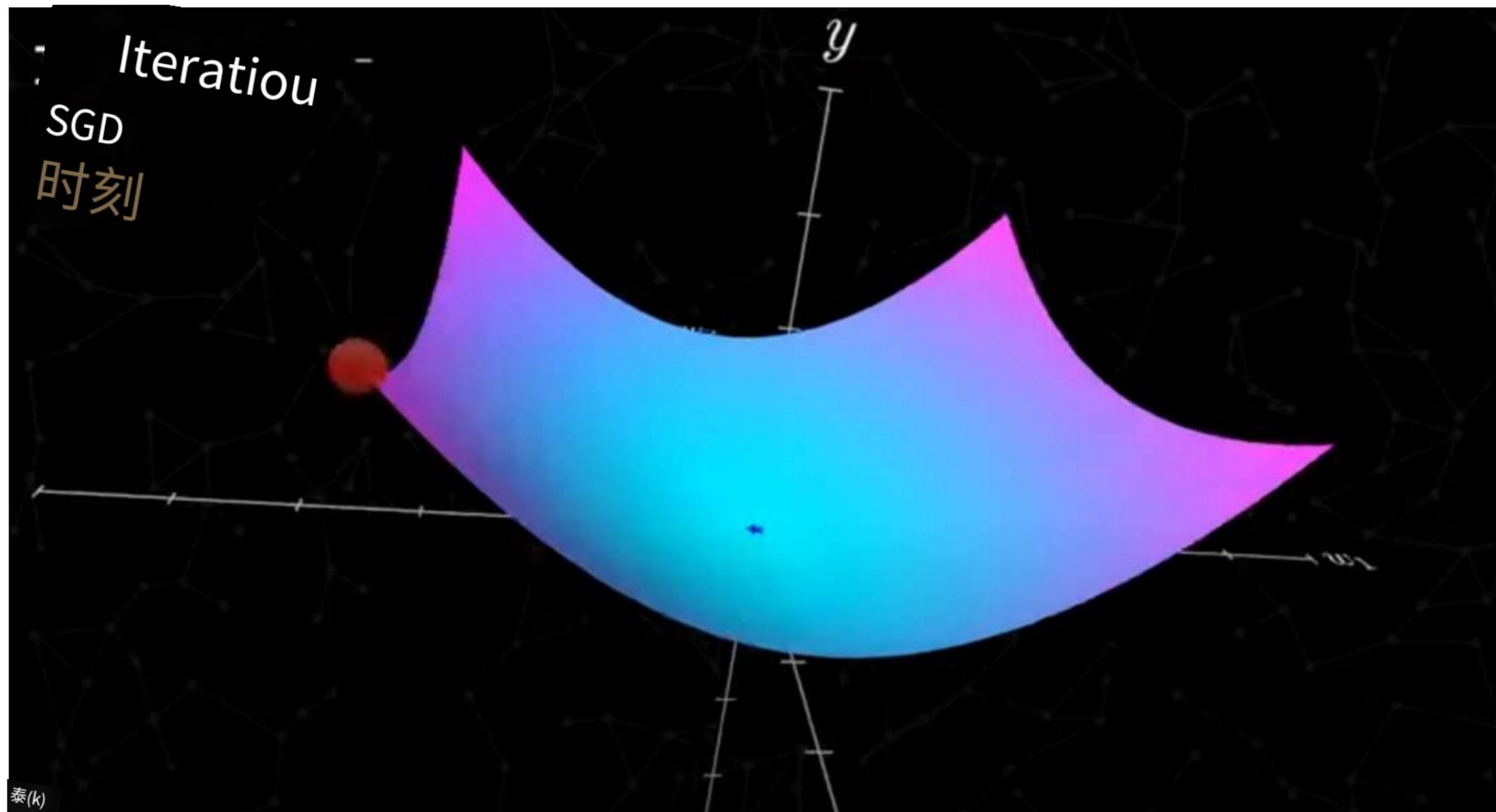
Momentum

$$\begin{cases} v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

- Ut: 速度矢量（过去梯度的加权平均）。
- β: 动量系数（通常为0.9）。
- α: 学习率。

优势	缺点
在浅层区域收敛更快。	如果β太高，可能会超调最小值。
减少陡峭山谷中的振荡。	需要调优β和α。

有了动量，它收敛得快得多。



Advantages	Disadvantages
More accurate gradient estimation.	Slightly more computation per step.
Reduces oscillations better than Momentum.	Still sensitive to hyperparameters.

2.2 Mathematical Formulation

- Update Rule:

“展望未来” 的梯度

$$\begin{cases} v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t + \beta v_{t-1}) \\ \theta_{t+1} = \theta_t - \alpha v_t \end{cases}$$

- The gradient is computed at $\theta_t + \beta v_{t-1}$ (future position).

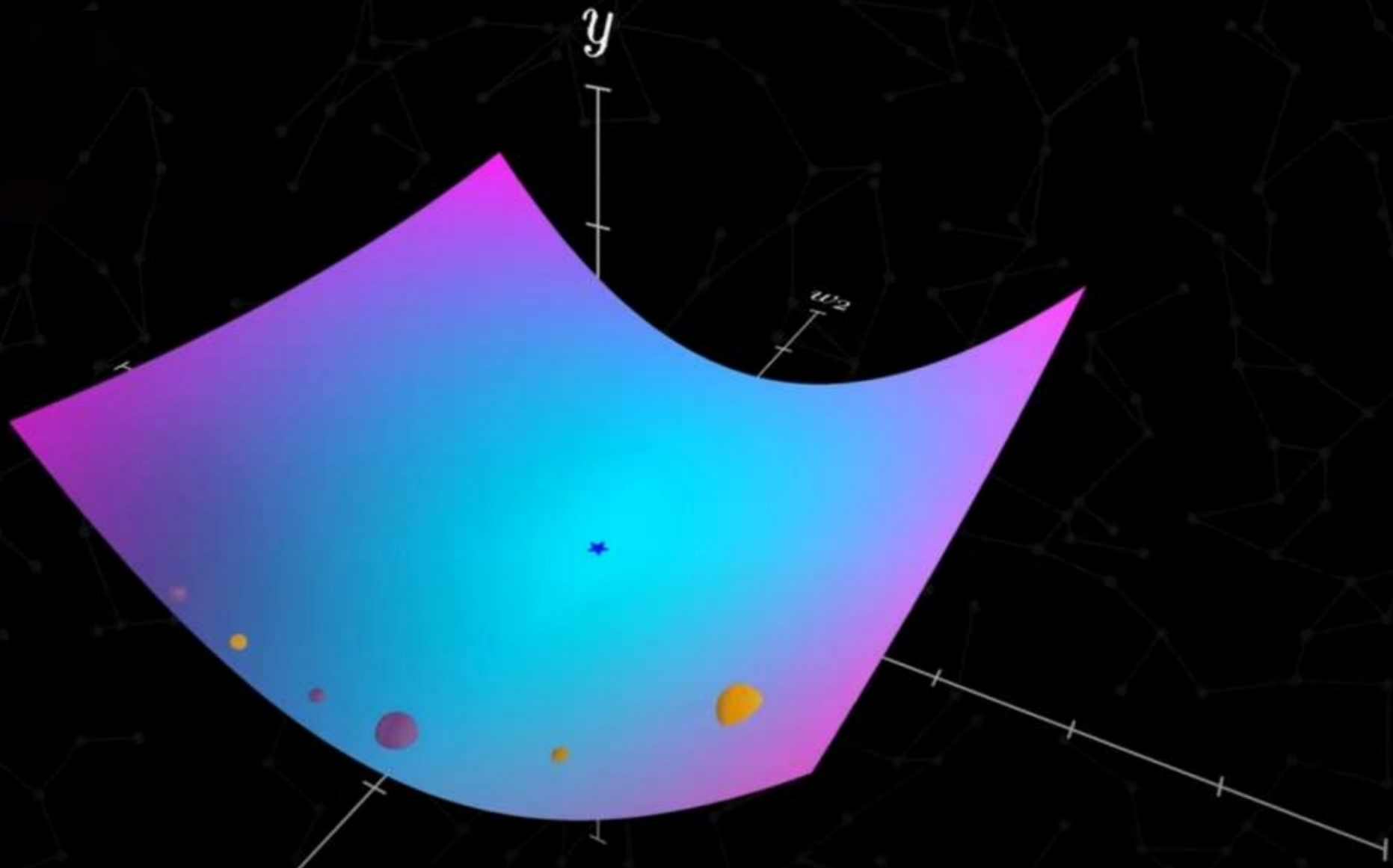
Advantages	Disadvantages
More accurate gradient estimation.	Slightly more computation per step.
Reduces oscillations better than Momentum.	Still sensitive to hyperparameters.

迭代# 5

故事

动力

玩(k)



Adagrad

Adagrad根据历史梯度大小调整每个参数的学习率。它将较小的更新分配给频繁特征和较大的更新分配给稀疏特征。

数学公式

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \nabla_{\theta} J(\theta_t)$$

“个性化”的学习率

- G_t : 过去梯度平方和的对角矩阵 ($G_t = \sum_{s=1}^t g_s g_s^T$)
- ϵ : 小常数 (例如, $1e-8$) , 以避免被零除。

关键属性

优势	缺点
适用于稀疏数据 (例如, NLP) 。	学习率可能会随着时间的推移而消失。
不需要手动调整学习率。	对于非凸问题表现不佳。

RMSProp

RMSProp通过用指数加权移动平均（EWMA）代替梯度的平方和来修复Adagrad的激进学习率衰减。

数学公式

$$\begin{cases} E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)(\nabla_{\theta} J(\theta_t))^2 \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla_{\theta} J(\theta_t) \end{cases}$$

- ：梯度平方的EWMA（ β 通常为0.9）。

添加指数加权移动平均方法

关键属性

优势	缺点
解决了Adagrad的消失LR问题。	仍然需要调整 β 和 a 。
对于非平稳目标（如rnn）效果很好。	对于某些任务可能需要动量。

亚当

它结合了动量（一阶矩）和均方根传播（二阶矩），以实现自适应学习率和平稳收敛。

Advantages	Disadvantages
Fast convergence and stability.	$\begin{cases} m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} J(\theta_t) & \text{(1st moment)} \\ v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} J(\theta_t))^2 & \text{(2nd moment)} \end{cases}$ tune $(\beta_1, \beta_2, \alpha)$.
Works well for deep learning (CNNs/Tr	
	$\begin{cases} \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} & \text{(bias correction)} \\ \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \odot \hat{m}_t \end{cases}$ ults are misused.

- β_1 : Momentum decay (default 0.9).
- β_2 : Squared gradient decay (default 0.999).

Key Properties

Advantages	Disadvantages
Fast convergence and stability.	More hyperparameters to tune $(\beta_1, \beta_2, \alpha)$.
Works well for deep learning (CNNs/Transformers).	May underperform if defaults are misused.

优化器	机制	何时使用	优势	弱点
动量	带速度的梯度平滑。	一般凸优化。	减少山谷中的振荡。	可能会低于最低限度。
NAG	在梯度步之前“向前看”。	高曲率地形。	比动量法收敛得更快。	每一步的计算量稍大一些。
Adagrad	逐参数自适应学习率（平方和）	稀疏数据（例如，自然语言处理）。	无需手动调整稀疏特征的逻辑回归参数。	LR 随时间逐渐消失。
RMSProp	平方梯度的指数加权移动平均值。	非平稳任务（例如，循环神经网络）	修复了 Adagrad 的学习率衰减问题。	需要调试 β 。
亚当	动量 + 均方根传播 + 偏差校正。	深度学习（默认选择）。	快速且稳定的收敛。	超参数敏感。



TED

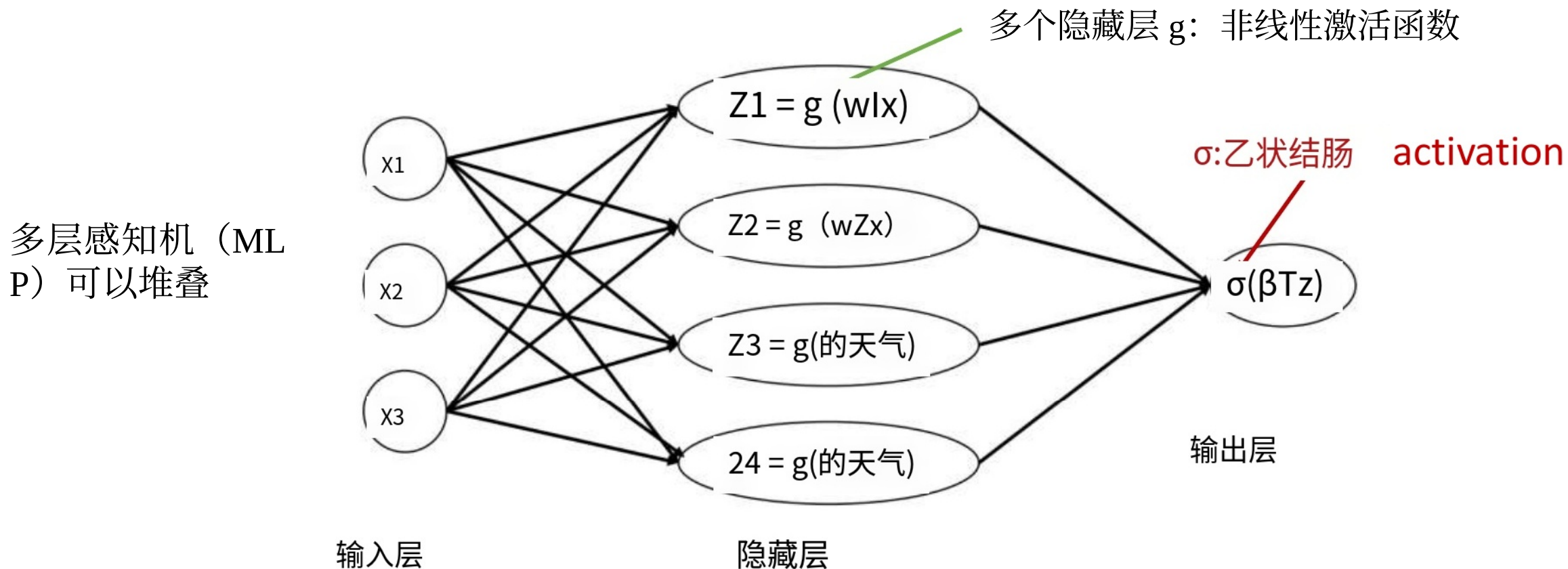
让我们把所有东西都整合在一起。



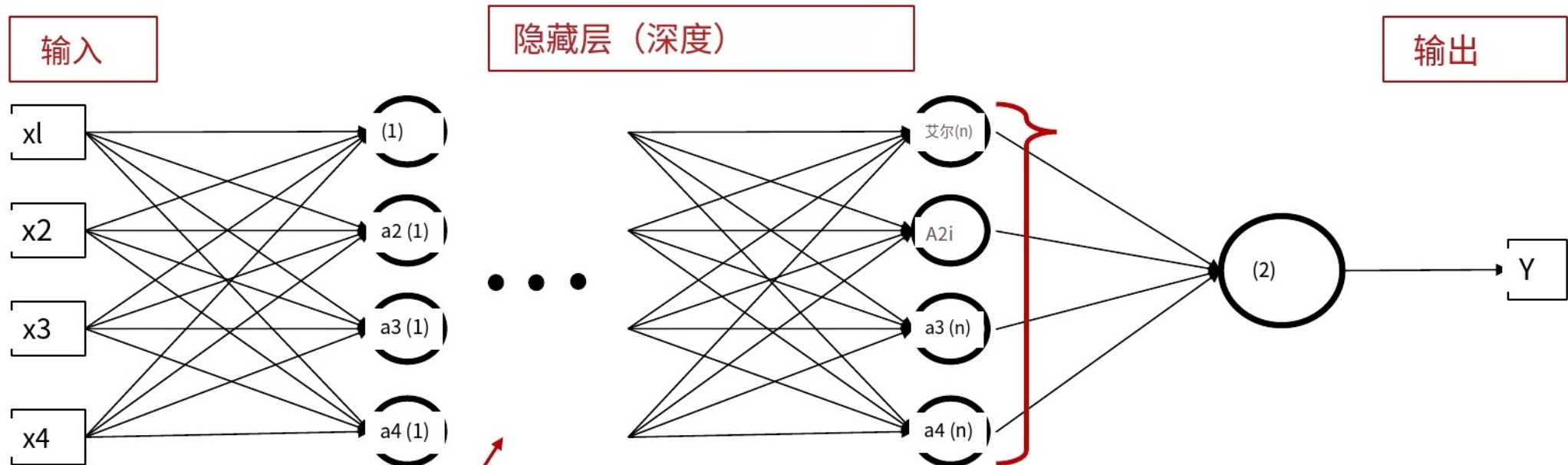
多层感知器

多层感知机由多层神经网络组成

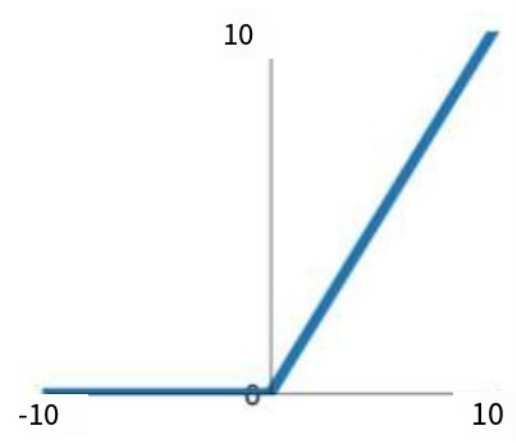
对于二分类问题: $f(x) = \sigma(\beta^T g(Wx)$



建筑学



隐藏层激活函数ReLU: $f(x) = \max(0, x)$



优化

损失函数最小化

回归：均方误差损失 $\frac{1}{N} \sum_{i=1}^N (f(\mathbf{x}_i) - y_i)^2$

分类：交叉熵损失 $\frac{1}{N} \sum_{i=1}^K H(f(\mathbf{x}_i), y_i)$

$$H(f(\mathbf{x}), y) = -(y \log f(\mathbf{x}) + (1 - y) \log(1 - f(\mathbf{x})))$$

正则化（避免过拟合）

$$\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|$$

L2

$$\min_{\mathbf{w}} L(\mathbf{w}) + \lambda \sum_i |w_i|^2$$

范数：应用广泛
也称为权重衰减



优化

神经网络权重初始化

- Xavier 初始化：保持多层之后的输出方差
- 凯明初始化：专为 ReLU 设计，当前默认选择

优化算法：梯度下降法

梯度下降算法

1. 用随机值初始化 $w(0)$

2. 对于 $t = 1, \dots, T$:

$$\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

优化器：ADAM



超参数调优与评估

典型的需要调整的超参数：正则化系数、学习率、层数（深度）、隐藏层维度（宽度）、批量大小……

拆分数据集：



评估指标：

$$\text{Accuracy} = \frac{TP + TN}{N}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

F1 分数, ROC 曲线下面积 (AUC)

1. 使用不同的超参数在训练数据上训练模型
2. 选择在验证数据上表现最佳的超参数
3. 报告模型在测试数据上的性能表现

训练多层感知机的挑战

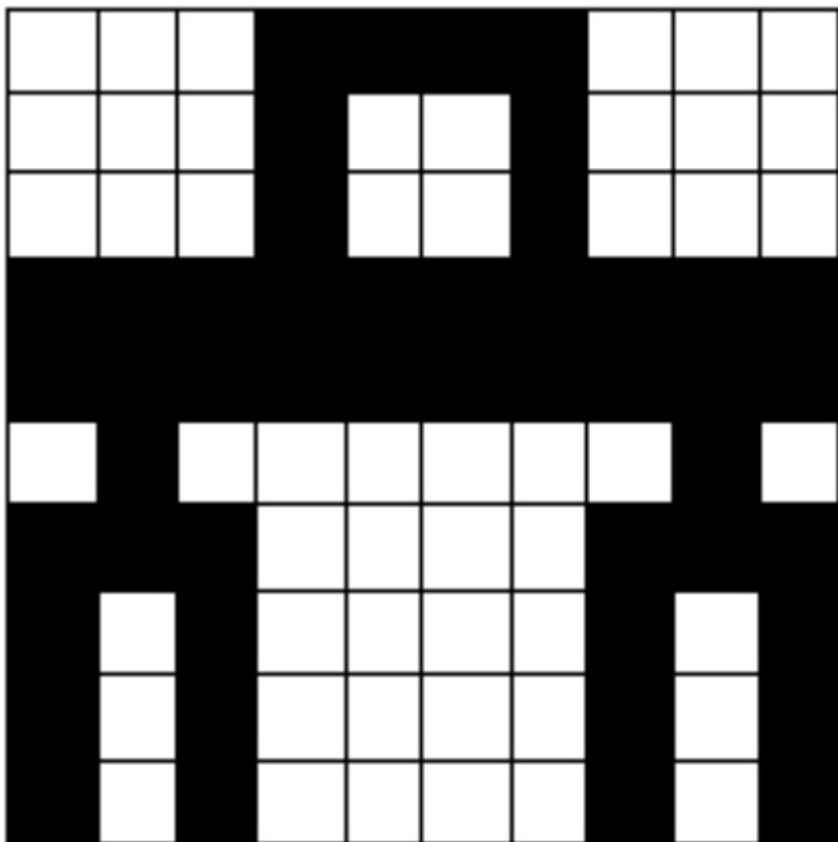
- 无封闭形式解，非凸优化
- 需要梯度下降法，这需要大量的计算
- 梯度消失/爆炸现象发生在深度神经网络中
 - 选择一个良好的初始化
 - 设置合适的优化器和超参数，尤其是学习率
 - 使用 ReLU 作为激活函数
 - 其他诸如 ResNet 之类的神经网络设计将在后面讨论。

卷积神经网络简介



计算机中图像是如何表示的？

黑白图像



矩阵表示

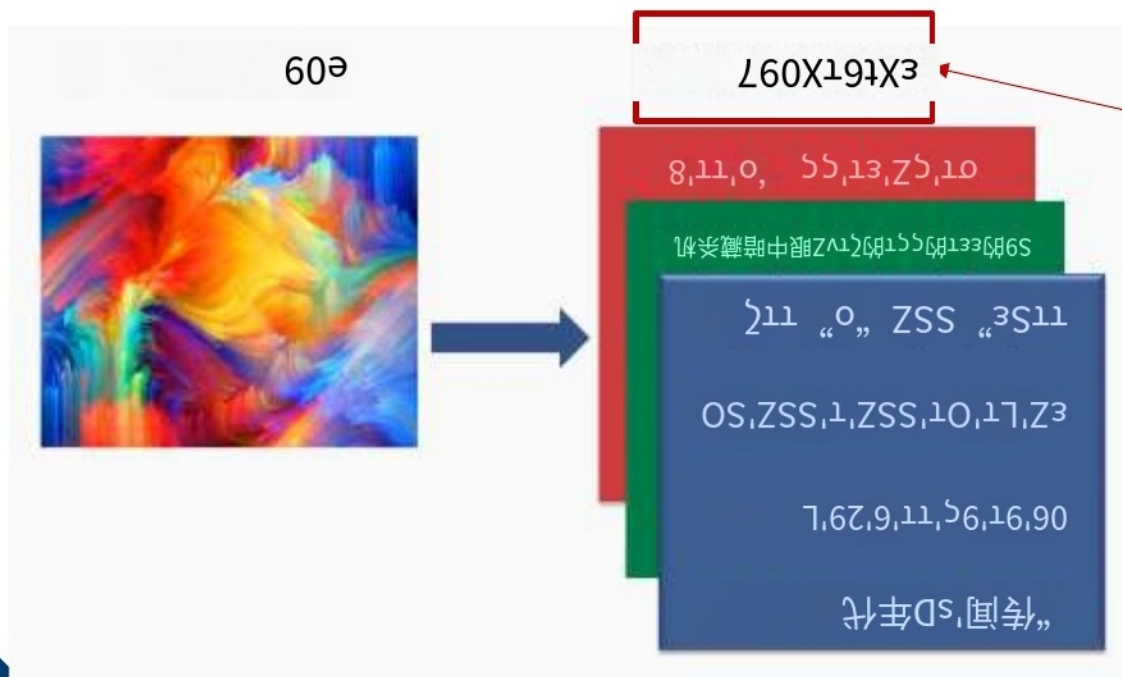
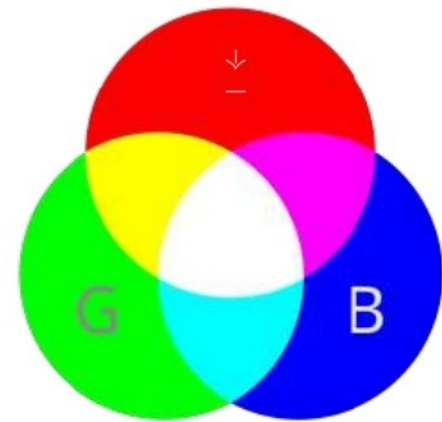


像素

0	0	0	1	1	1	1	0	0	0
0	0	1	1	0	0	1	0	0	0
1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1
0	1	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	1	1	1
1	0	0	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	0	1
1	0	0	0	0	0	0	1	0	1

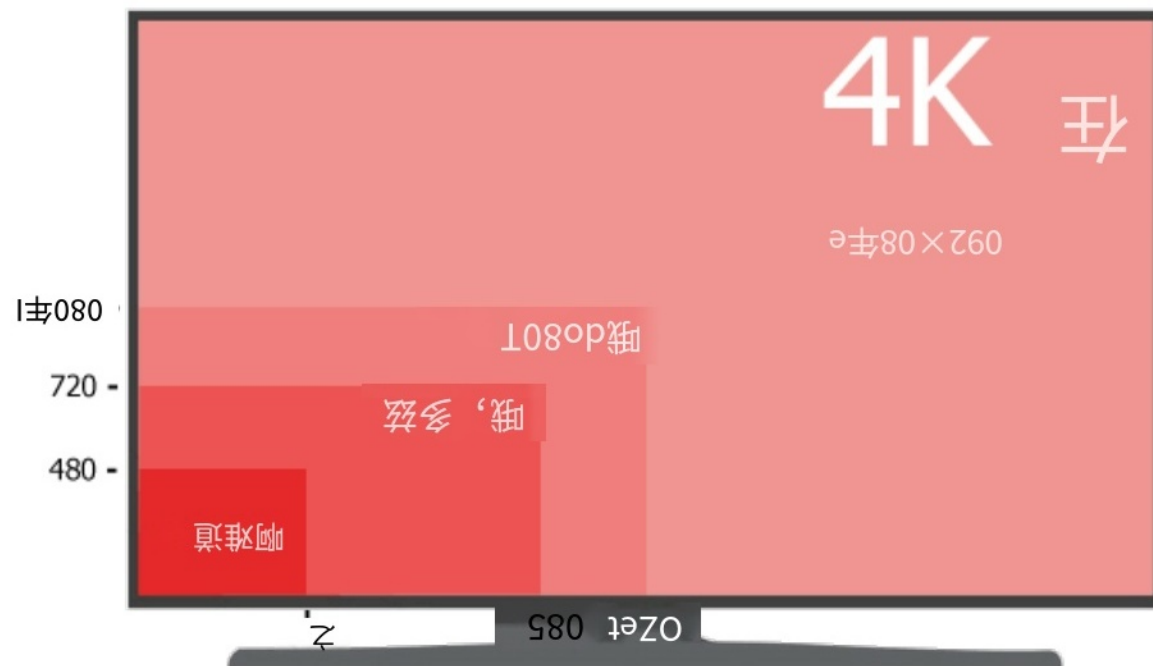
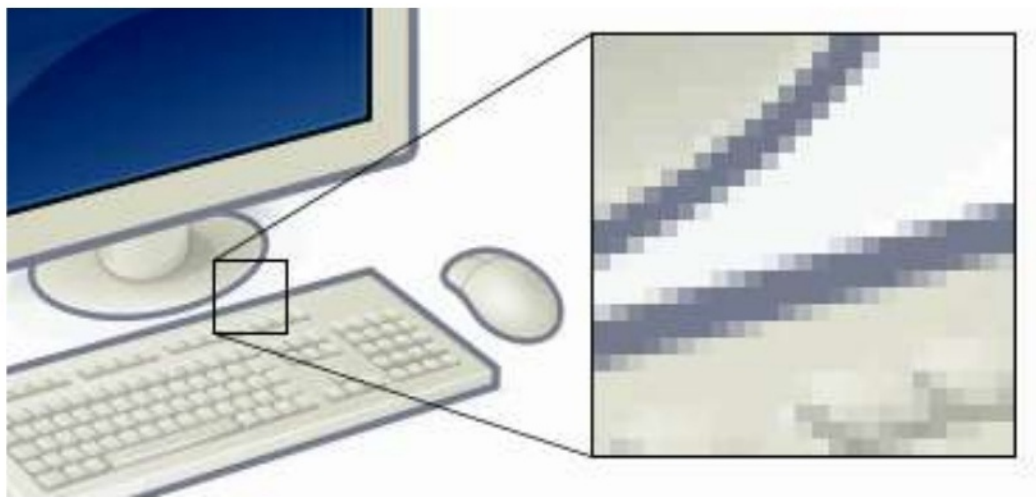
计算机中图像是如何表示的？

红、绿、蓝可以组合成任何颜色。



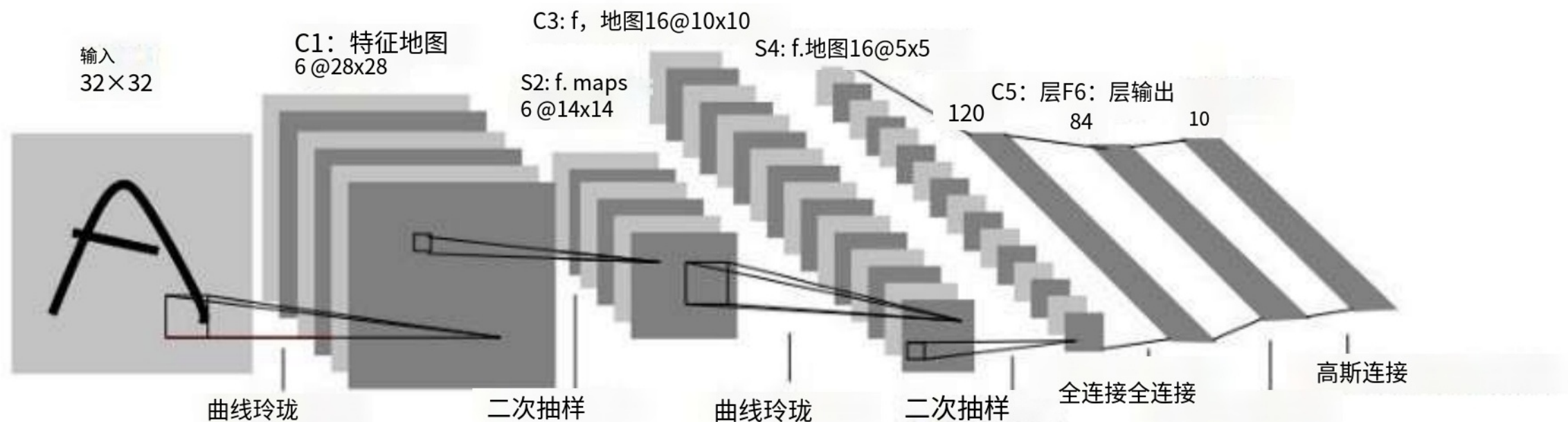
宽度：260 像素
高度：194 像素
通道：3（RGB）

计算机中图像是如何表示的?



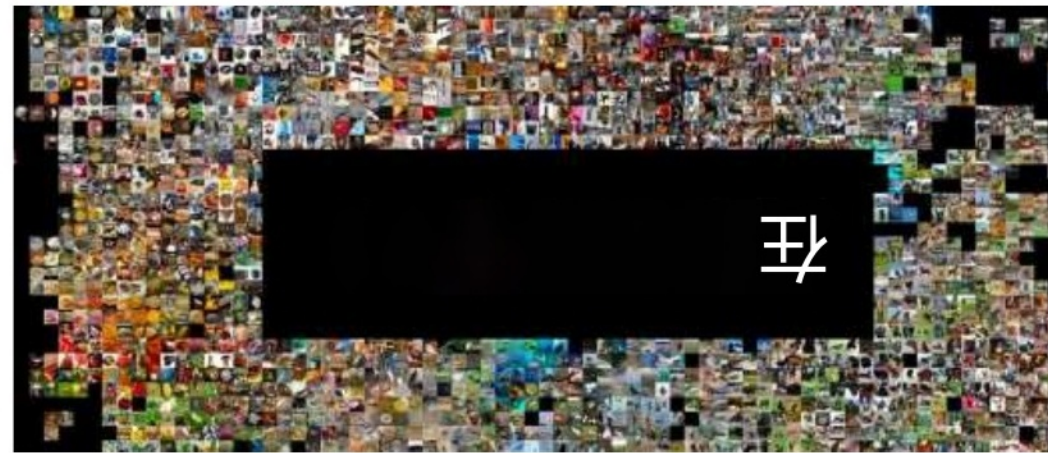
CNN的历史

基于梯度的学习在文档识别中的应用 [LeCun、Bottou、Bengio、Haffner 1998]

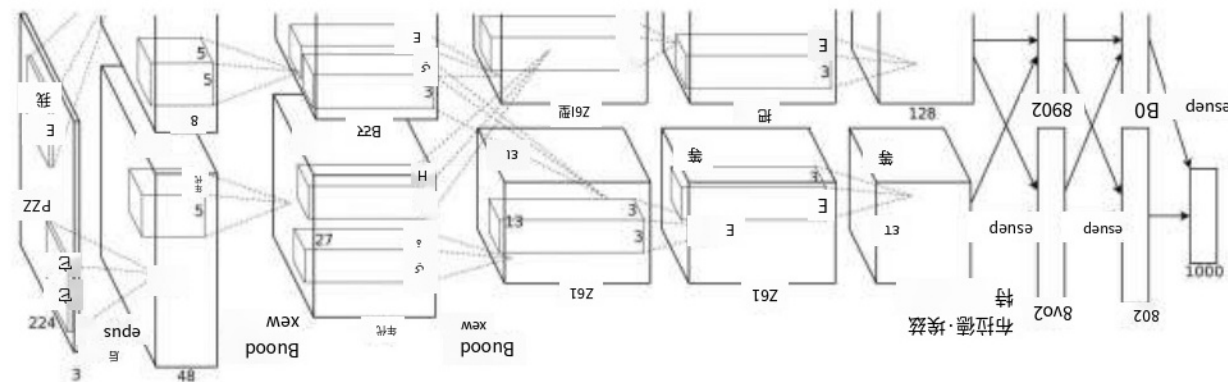


深度学习的开端

李飞飞创建了 ImageNet：当时最大的图像分类数据集



基于深度卷积神经网络的 ImageNet 分类
[Krizhevsky、Sutskever、Hinton, 2012]

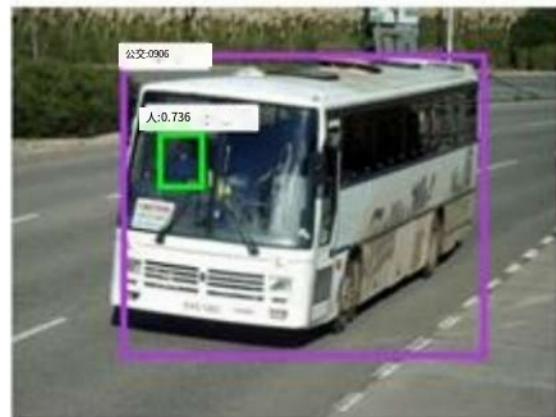
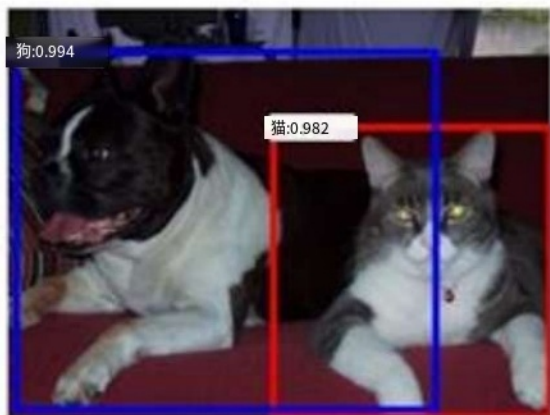
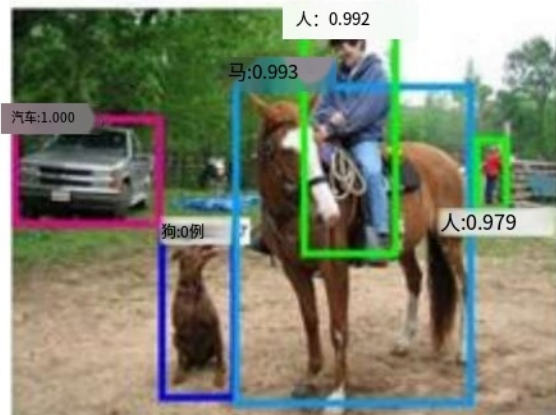


AlexNet

2012 – 2020: 卷积神经网络在所有视觉任务中占据主导地位

检测

检测



图片版权所有任少卿, 开明, Ross Girschick, 孙健, 2015. 经许可转载。

图片版权归Clement Farabet所有, 2012转载已获许可。

[Farabet等, 2012]

更快R-CNN: 任少卿, 开明, 格希克, 孙健2015]



2012 – 2020: 卷积神经网络在所有视觉任务中占据主导地位

图像字幕



一只坐在草地上的白色泰迪熊



一个穿着棒球服的男人在投球



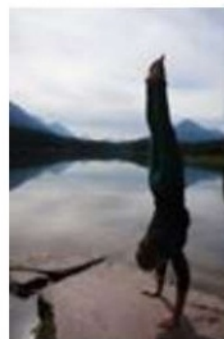
一个女人手里拿着一只猫



一个人在冲浪板上乘风破浪



一只猫坐在地板上的手提箱上



一个拿着冲浪板站在沙滩上的女人

所有图片均为recco公有领域：
<https://pixabay.com/en/luggage-antique-cat-1643010/>
<https://pixabay.com/en/teddy-plush-bears-cute-teddy-bear-1623436/>
<https://pixabay.com/en/surf-wave-summer-sport-litoral-1668716/>
<https://pixabay.com/en/woman-female-model-portrait-adult-983967/>
<https://pixabay.com/en/handstand-lake-meditation-496008/>
<https://pixabay.com/en/baseball-player-shortstop-infield-1045263/>

说明文字由Justin Johnson使用NeuralTalk2生成

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]

2012 - 2020: 卷积神经网络在所有视觉任务中占据主导地位

文本到图像生成

罗姆巴赫等人, 《基于潜在扩散模型的高分辨率图像合成》, CVPR 202



一幅毕加索风格的僵尸画

一只半老鼠半章鱼的图像

一幅画着一只松鼠正在吃东西的画

汉堡包

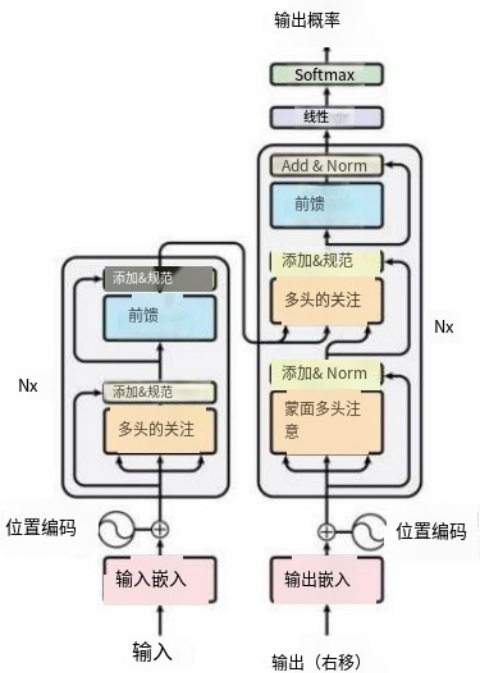
一幅水彩画, 画
中是一把看起来像的
椅子

章鱼

一件印有
铭文: “我喜欢生成
模型!”

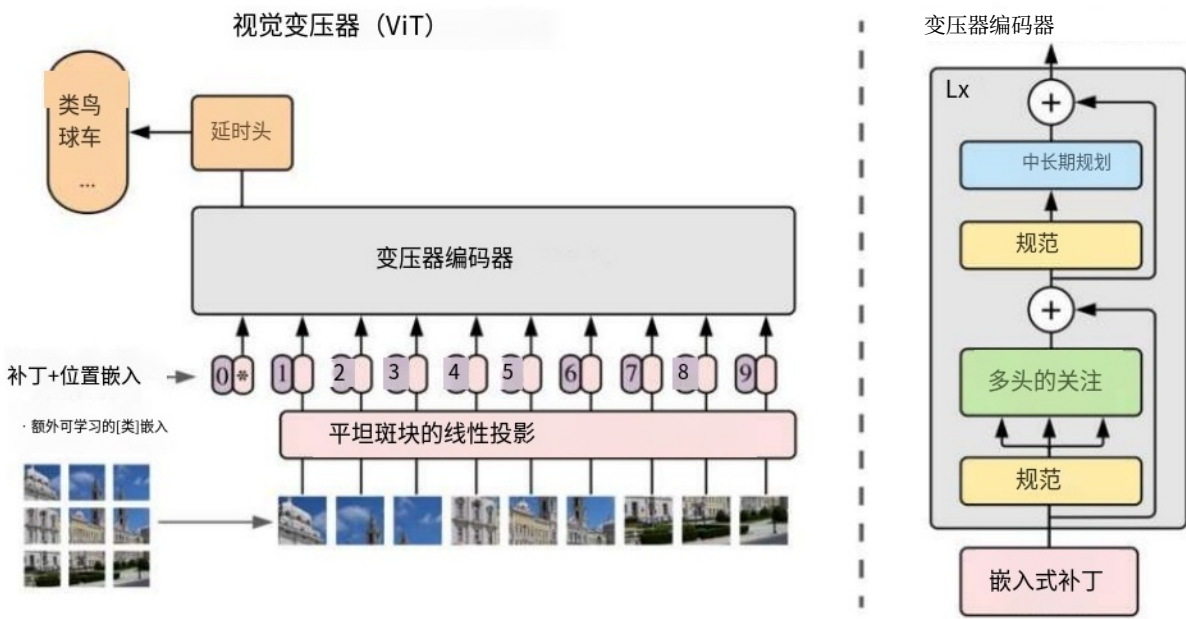
2021 - 至今：变压器已占据主导地位

2017：变形金刚
语言任务



瓦斯瓦尼等人，《注意力机制全搞定》，NeurIPS 2017

2021：变形金刚
用于视觉任务



多索维斯基等人，“一幅图像抵得上 16×16 个词：用于图像处理的 Transformer 模型”

《大规模识别》，ICLR 2021

